

CS5344 Project Proposal

Steering a Tabular Diffusion Model Away from Attacks

Synthetic normal traffic for anomaly detection on NSL-KDD and UNSW-NB15

Group 09

Name	Student number
Brian Kheng	A0290248B
Mei Xianle	A0331789N
Passion Goh Wei Ling	A0222740R

27 September 2026

1 Introduction

For each of two network intrusion datasets we get a labelled training set and have to submit a fixed number of synthetic normal rows: 40,000 for NSL-KDD [14] and 70,000 for UNSW-NB15 [11]. These samples are used to train predefined anomaly detectors, ECOD and Isolation Forest, which are evaluated using AUPRC on hidden validation and test sets.

The hidden sets also contain attack types that are not in the training data, so copying the training distribution as closely as possible is not the goal. We want synthetic normal rows that cover legitimate traffic well but stay out of the regions where attacks sit.

Our plan is to train the mixed-type diffusion model TabDiff [13] on all training rows, with the label as an extra column, and then sample normal rows with a guidance term that pushes each sample away from what the model associates with attacks. Section 2 looks at where the attacks actually sit in the data, Section 3 covers related work, and Section 4 presents our proposed approach and evaluation strategy.

2 Exploratory data analysis

Table 1 summarises the two training sets. Both are mixed-type tables with three categorical columns, and both have roughly two normal rows for every attack. The numeric columns are hard to model. In NSL-KDD, 23 of the 38 numeric columns are zero in over 90% of normal rows, and `src_bytes` has a skewness of about 129 among normal rows. Eleven NSL-KDD columns and ten UNSW-NB15 columns take between 2 and 20 distinct values (flags, TTLs and small counters), and two NSL-KDD columns, `num_outbound_cmds` and `is_host_login`, are constant. A generator that treats every numeric column as continuous will therefore produce values that never occur in the data, such as a TTL of 61.7 or a `logged_in` of 0.4.

Table 1: The two training sets. Ranges and categories are taken from the normal training rows. Computed by `proposal/eda.py`.

	NSL-KDD	UNSW-NB15
Training rows (normal / attack)	20,000 / 10,000	35,000 / 15,000
Required synthetic rows N	40,000	70,000
Numeric / categorical features	38 / 3	39 / 3
Numeric features with 2 to 20 distinct values	11	10
Constant features	2	0
Numeric features that are zero in over 90% of normal rows	23	4
Attacks with a category no normal row uses	32.5%	1.7%
Attacks with a numeric value outside the normal range	2.0%	1.1%
Attacks inside the normal range on every feature	65.5%	97.2%
Numeric features with normal-vs-attack KS distance above 0.5	13	3

The briefing describes two kinds of anomaly: an odd value far from its usual range, and a rare combination of values that each look normal on their own. To see which kind the training attacks are, we checked for each attack row whether any numeric value falls outside the range seen in normal rows, or whether any categorical value never appears in a normal row. In NSL-KDD, 32.5% of attacks use a category that no normal row uses (41 of the 67 `service` values appear only in attacks), but only 2.0% have an out-of-range number. The other 65.5% sit inside the normal range on every column. UNSW-NB15 is more extreme: 97.2% of its attacks are inside the normal range on every column, so for almost all of them there is no single odd value to catch.

Figure 1 ranks the numeric columns by the Kolmogorov-Smirnov (KS) distance between their normal and attack values. NSL-KDD has 13 columns above 0.5, led by the byte counts, while UNSW-NB15 has three, all derived from TTLs. Almost every UNSW-NB15 attack (99.9%) has `sttl`

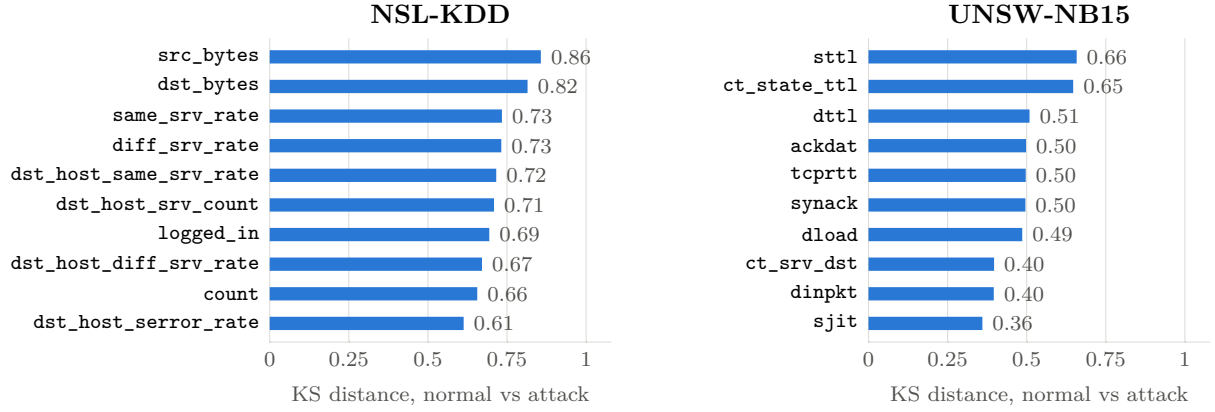


Figure 1: The ten numeric columns of each training set whose normal and attack values differ most, measured by the two-sample Kolmogorov-Smirnov (KS) distance. Separation in UNSW-NB15 is weaker and concentrated in the TTL columns.

equal to 254 (71.5%) or 62 (28.5%), and both values are common in normal traffic too, at 27.1% and 7.0% of normal rows. So what gives these attacks away is how often those values occur and what they occur alongside, not the values themselves.

3 Related work

3.1 Unsupervised anomaly detection

Unsupervised anomaly detection suits network intrusion data, because labelled attacks are scarce and new attack types keep appearing. Chandola et al. [1] survey the classical methods, and Pang et al. [12] review deep learning approaches. The two detectors used in this project, Isolation Forest [9] and ECOD [8], are both fast, unsupervised methods for tabular data. Isolation Forest scores a row by how few random splits it takes to isolate it, and ECOD scores it by how far into the tails of each column’s empirical distribution its values fall.

The two detectors assume different things about what makes a row anomalous. ECOD scores each column on its own and adds up the tail probabilities, so it can’t see an unusual combination of normal values, which is exactly the kind of attack Section 2 found most of. ADBench [5], which compares 30 detectors on 57 datasets, finds that no unsupervised detector wins across the board and that performance depends on how well a detector’s assumptions fit the anomalies in the data. Here the detectors are fixed, so we can’t pick or tune them. What we control is the normal data they are fitted on.

3.2 Tabular data synthesis

The simplest option is SMOTE [2], which creates a new row on the line between a row and one of its nearest neighbours. It is cheap, but interpolation only makes sense for numeric columns and never moves far from the existing rows. CTGAN and TVAE [15] model mixed-type tables with a GAN and a variational autoencoder, using mode-specific normalisation for multi-modal numeric columns and a conditional generator so that rare categories still get sampled. TabDDPM [7] brought diffusion models to tables, with Gaussian diffusion for numeric columns and multinomial diffusion for categorical ones. TabSyn [16] instead runs score-based diffusion in the latent space of a VAE. TabDiff [13] trains a single continuous-time diffusion over both types, with masking diffusion for categorical columns and a learnable noise schedule for every column, and reports better fidelity than these earlier models on the benchmarks it tests.

These methods are usually judged on fidelity, meaning how closely the synthetic data matches

the real distribution, and on utility, meaning how well a model trained on synthetic data does on real data [4]. Our utility measure is unusual. The downstream models are two fixed unsupervised detectors, and part of the test set is attack types nobody has seen, so the most faithful copy of the training data is not necessarily the most useful one.

3.3 Guidance in diffusion models

Diffusion models can also be steered while sampling. Classifier guidance [3] adds the gradient of a separately trained classifier to the model’s score. Classifier-free guidance [6] drops the classifier: one network is trained with and without the condition, and sampling extrapolates from the unconditional prediction towards the conditional one, trading some diversity for samples that fit the condition more strongly. Liu et al. [10] showed that the same arithmetic can combine several conditions, including negated ones, where subtracting the prediction for an unwanted concept moves samples away from it. That work was on images and text prompts, but the arithmetic doesn’t depend on the data type. With *normal* and *attack* as the two conditions, negating *attack* should push samples out of the regions attacks occupy while keeping them normal.

4 Intended approach

Figure 2 shows the generator. It is one pipeline for both datasets, with only the guidance weight w , the training length and N set per dataset.

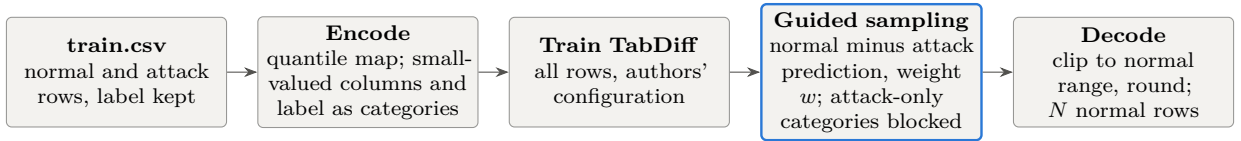


Figure 2: The generator. The outlined step is our addition to TabDiff.

4.1 Backbone

We use the TabDiff authors’ model code and default configuration unchanged, and write the data handling, training loop and guidance ourselves. Before training, numeric columns with at most 20 distinct values are treated as categories, so the model can only produce values that actually occur, and constant columns are held fixed. Continuous columns are mapped to a standard normal with a quantile transform, following the TabDDPM and TabDiff preprocessing, which deals with the heavy tails from Section 2. The label is added as one more categorical column and the model is trained on every training row, attacks included, so that it learns what attacks look like as well as normal traffic. The guidance needs both.

Training follows the authors’ trainer: 8,000 epochs of AdamW with batch size 4,096, a plateau learning-rate decay, a continuous-loss weight that is annealed over training, and an exponential moving average of the weights, from which we keep the copy with the lowest training loss in the second half of training.

4.2 Contrastive guidance

At every sampling step we run TabDiff’s denoiser D twice on the current noisy row x , once with the label set to normal and once with it set to attack, and combine the two outputs:

$$\tilde{D}(x) = D(x \mid \text{normal}) + w (D(x \mid \text{normal}) - D(x \mid \text{attack})). \quad (1)$$

The same combination is applied to the logits of the categorical columns. With $w = 0$ this is plain label-conditional TabDiff. With $w > 0$ it is classifier-free guidance with the attack condition in place of the unconditional one, which is a negated condition in the sense of Liu et al. [10].

The briefing asks us to explain the distributional properties of the generated data, and (1) gives a direct answer. The model’s score is a linear function of the denoiser output, so (1) follows the score of

$$p_w(x) \propto p(x \mid \text{normal}) \left(\frac{p(x \mid \text{normal})}{p(x \mid \text{attack})} \right)^w \quad (2)$$

at each noise level. The samples follow the normal distribution, reweighted by how strongly each region favours normal over attack. Regions only normal traffic uses keep their mass or gain some, while regions shared with attacks, such as `stt1` = 254 in UNSW-NB15, are thinned out, and w sets how hard. As with classifier-free guidance, (2) is exact only for the noisy intermediate distributions, not for the final samples, so we also measure the final samples directly (Section 4.4).

Two further constraints keep every sample a valid normal row. Categories that no normal training row uses, including the attack label itself, are blocked during sampling by setting their logits to $-\infty$, so, for example, none of the 41 attack-only NSL-KDD services ever appears in the submission. After sampling, continuous values are clipped to the range seen in normal rows and rounded to each column’s precision, which keeps the output within the submission’s types and ranges.

4.3 Why this design

The backbone has to cope with mixed types, heavy tails and many small-valued columns, and it has to learn how columns occur together, because Section 2 found that this is where most attacks differ from normal traffic. TabDiff models the categorical and numeric columns in one joint diffusion instead of two separate processes, and its per-column noise schedule lets a column that is almost always zero and a heavy-tailed byte count be denoised at different rates. Of the generators in Section 3, it also reports the best fidelity on most of the authors’ benchmarks.

Being a diffusion model in data space matters for the guidance too. CTGAN and TVAE can condition on a category, but they have no score to combine, so there is no direct way to push their samples away from a condition. TabSyn could be guided, but its diffusion runs on VAE latents, so (2) would describe latent codes, not columns, and the result would be harder to explain in terms of the data.

Guidance itself follows from the UNSW-NB15 findings. When 97.2% of attacks are inside the normal range, clipping out-of-range values does almost nothing, and a per-column adjustment can’t tell an `stt1` of 254 in normal traffic from the same value in an attack. A label-conditioned model can, because it sees the other columns of the row as well. An earlier version of our generator did a per-column version of this by hand, reshaping each column of a bootstrap sample away from attack values. Contrastive guidance is the learned version, acting on all columns jointly, and we keep the hand-written one as a baseline.

4.4 Evaluation plan

We reimplemented the scoring described in the briefing as a local judge: ECOD from PyOD and scikit-learn’s Isolation Forest with 500 trees for each of the seeds 0 to 9, both fitted on the submitted rows with one-hot encoded categories, and the mean of the ECOD AUPRC and the average Isolation Forest AUPRC. Some details are unspecified, such as the Isolation Forest subsample size and the categorical encoding, so we record our choices and treat the judge as a proxy that is good for comparing our own variants, not for predicting leaderboard scores.

The full public releases of both datasets have plenty of labelled rows left over once the coursework training rows are removed. The hidden sets are drawn from the same datasets, though, so these rows may overlap them, and we won’t use them to choose hyperparameters. To tune w we will hold

out a stratified 20% of `train.csv`, train the generator on the other 80%, and score on the holdout. The public data then serves only as a final check.

Guidance pushes away from the attacks the model has seen, while the hidden test set has types it hasn't. The attack type isn't released, so we will simulate unseen types by clustering the training attacks with k -means on the encoded features, leaving one cluster out of generator training at a time, and scoring on the holdout normal rows plus the held-out cluster. If guidance helps on seen attacks but hurts on held-out clusters, we will lower w or mix guided and unguided samples.

For every run we will report the two detector AUPRCs separately, per-column KS and total variation distances to the normal training rows, and the mean attack probability that a normal-versus-attack classifier fitted on `train.csv` gives the samples. The last one checks directly that guidance pushes in the intended direction, and together they let us explain score changes in terms of the data. The ablations are $w \in \{0, 0.5, 1, 2, 4\}$ and sampling with and without blocked categories. The baselines are a bootstrap of the normal rows, CTGAN, TVAE and TabDDPM trained on the normal rows with their published settings, and our earlier hand-written reshaping. We will submit to Kaggle sparingly, mainly to confirm that local improvements carry over, since the public leaderboard only covers the validation attack types and not the extra ones in the test set.

The main risk is over-guidance. A large w also removes normal traffic that happens to look like attacks, and those normal rows in the hidden set then score as anomalies, which costs precision. The holdout and held-out-cluster experiments are there to find where that starts. Training is also slow, about two hours for NSL-KDD on a laptop GPU and longer for UNSW-NB15, but the trained model is cached, so each new value of w only reruns sampling, which takes minutes.

References

- [1] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 2009.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 2002.
- [3] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [4] Cristóbal Esteban, Stephanie L. Hyland, and Gunnar Rätsch. Real-valued (medical) time series generation with recurrent conditional GANs. arXiv:1706.02633, 2017.
- [5] Songqiao Han, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. ADBench: Anomaly detection benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.
- [6] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.
- [7] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling tabular data with diffusion models. In *International Conference on Machine Learning (ICML)*, 2023.
- [8] Zheng Li, Yue Zhao, Xiyang Hu, Nicola Botta, Cezar Ionescu, and George H. Chen. ECOD: Unsupervised outlier detection using empirical cumulative distribution functions. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12181–12193, 2023.
- [9] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *IEEE International Conference on Data Mining (ICDM)*, 2008.

- [10] Nan Liu, Shuang Li, Yilun Du, Antonio Torralba, and Joshua B. Tenenbaum. Compositional visual generation with composable diffusion models. In *European Conference on Computer Vision (ECCV)*, 2022.
- [11] Nour Moustafa and Jill Slay. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *Military Communications and Information Systems Conference (MilCIS)*, 2015.
- [12] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton van den Hengel. Deep learning for anomaly detection: A review. *ACM Computing Surveys*, 54(2), 2021.
- [13] Juntong Shi, Minkai Xu, Harper Hua, Hengrui Zhang, Stefano Ermon, and Jure Leskovec. TabDiff: a mixed-type diffusion model for tabular data generation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [14] Mahbod Tavallaei, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009.
- [15] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [16] Hengrui Zhang, Jiani Zhang, Balasubramaniam Srinivasan, Zhengyuan Shen, Xiao Qin, Christos Faloutsos, Huzefa Rangwala, and George Karypis. Mixed-type tabular data synthesis with score-based diffusion in latent space. In *International Conference on Learning Representations (ICLR)*, 2024.